

Seminar Report

On

Google Cloud

Generative AI

Google Cloud Generative AI

Partial report in partial fulfillment of the requirement of the Degree of

Bachelor of Technology

In

Computer Science and Engineering

By

Name of the Student : Kotte Yashvitha

Regd. Number : 22541A0548

Under the supervision of

Mr.Bharat Veerla

M.Tech, (Ph.D) Associate Professor,

Head of the Department of CSE, CSE (AI&DS) To



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING SRI
SARATHI INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Approved by AICTE Affiliated to JNTUK, ISO9001:2008 Certified)
NUZVID521201,ELURU District, A.P.
2022-2026**

CERTIFICATE

Candidate Declaration

I declare that the work presented in this report 'Full Stack Web Development' in partial fulfillment of the requirements for the award of the degree of bachelor of Technology in Computer Science and Engineering submitted in the Sri Sarathi institute of Engineering and Technology of department of Computer Science and Engineering in partial fulfillment of requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering to the Jawaharlal Nehru Technological University, Kakinada is a record of bonafide work carried out by him/her under our guidance and supervision of **Mr. VEERLA BHARAT, M.Tech, (PhD), in Sri Sarathi Institute Of Engineering And Technology, Nuzvid.**

The matter embodied in the report has not been submitted for the award of any other degree or diploma. It contains sufficient information to describe the various tasks performed by me during the internship.

Signature of the Student

ACKNOWLEDGEMENT

We thank our honorable Chairman **Sri. S.V. SUDHEER Garu**, for his moral support and the excellent facilities provided. We are grateful to **Sri. S. SRIDHAR Garu**, Principal, **Sri Sarathi Institute of Engineering & Technology, Nuzvid**, for his support throughout the project for given the permission to undergo the project work.

We would also like to thank **Sri MR. VEERLA BHARAT, M.Tech (PhD)** Head of the **Department of Computer Science Engineering and Technology** for given us the opportunity to complete the project successfully.

We sincerely thanks to **Mr.VEERLA BHARAT M.Tech (PhD)**,**Sri Sarathi Institute of Engineering and Technology, Nuzvid** for his valuable guidance and support.

We would like to thank all the people who have helped us in completion of project, credit goes to all my team members and our friends those who shows constant encouragement till the completion of this interesting project within the limited time.

I thank for timely suggestions and constant encouragement that boosted up our moral and let to the accomplishment of this project. Firstly, we indebted to my parents, friends and relatives for the physical, psychological and moral extended by them at all time.

We make this project not only for marks but also to increase our knowledge.

AUTOSAGE APP USING GEMINI FLASH

AI-Powered Vehicle Information & Advisory System

Category: Google Cloud Generative AI

Team ID: LTVIP2026TMIDS64348

Team Size: 4

Team Members

Kotte Yashvitha – Team Leader

Sandhya Pallagani – Team Member

Kolluru Teja Mahitha – Team Member

Gandikota Kalyan Kumar – Team Member

Year: 2026

Table of Content

- 1. Introduction & Project Overview**
- 2. Project Objectives**
- 3. Technologies Used**
 - 3.1 Python
 - 3.2 Streamlit
 - 3.3 Google Generative AI (Gemini Flash)
 - 3.4 Python-Dotenv
 - 3.5 Pillow (PIL)
- 4. System Architecture**
- 5. Project Structure**
- 6. Requirements Specification**
- 7. Google API Key Setup**
- 8. Model Integration (Gemini Flash)**
- 9. Image Processing Function**
- 10. Prompt Engineering**
- 11. Web Interface (Streamlit)**
- 12. Running the Application**
- 13. Model Deployment**
- 14. Key Features**
- 15. Future Enhancements**
- 16. Conclusion**

LIST OF FIGURES

- Fig 1.1 – Gemini AI Interface Image
- Fig 3.1 – Technologies Used Overview
- Fig 3.2 – Python Code Example
- Fig 3.3 – Streamlit Interface Code
- Fig 3.4 – Gemini Flash Model Integration Diagram
- Fig 4.1 – System Architecture Diagram
- Fig 4.2 – Gemini API Architecture
- Fig 5.1 – Project Structure Diagram
- Fig 6.1 – Requirements Specification Chart
- Fig 8.1 – Model Integration Workflow
- Fig 8.2 – Gemini Flash Processing Diagram
- Fig 9.1 – Image Processing Function Diagram
- Fig 10.1 – Prompt Engineering Diagram
- Fig 10.2 – Prompt Design Workflow
- Fig 11.1 – Streamlit Web Interface
- Fig 12.1 – Running Application Terminal Output
- Fig 13.1 – Deployment Workflow Diagram
- Fig 15.1 – Future Enhancements Diagram
- Fig 16.1 – Conclusion Illustration

1. INTRODUCTION & Project Overview

Artificial Intelligence (AI) has rapidly evolved in recent years, especially with the development of **Generative AI models** that can understand and generate human-like responses. These models are capable of analyzing user input, understanding context, and providing meaningful outputs. Google's **Gemini** is one such advanced model capable of processing both text and visual inputs, making it suitable for building intelligent and interactive applications.

The **AutoSage App using Gemini Flash** is designed as an AI-powered vehicle advisory system that helps users obtain detailed and structured information about automobiles. Users can interact with the system by asking vehicle-related questions or by uploading vehicle images. The application processes this input using the **Gemini Flash model** and provides useful information such as vehicle specifications, key features, mileage, maintenance tips, and approximate price range. This makes it easier for users to access vehicle-related information quickly without searching through multiple sources.

The application is developed using **Python** and **Streamlit**, which allows the creation of a simple and user-friendly web interface. Streamlit helps in building interactive applications where users can upload images, enter queries, and receive AI-generated responses instantly. By combining Generative AI with web development, this project demonstrates how modern AI technologies can be applied to solve real-world problems and improve user experience.

The **AutoSage App** is an AI-powered vehicle advisory system developed as part of the **Google Cloud Generative AI Virtual Internship**. It uses **Google's Gemini Flash model** to provide intelligent responses to vehicle-related questions and uploaded vehicle images.

Users can:

- Ask vehicle-related questions
- Upload images of vehicles
- Receive structured information like specifications, features, mileage, maintenance tips, and price range

2. Project Objectives

The main objectives of the **AutoSage App using Gemini Flash** project are:

- To design and develop an **AI-powered vehicle information and advisory system**
- To integrate **Google Gemini Flash** for processing both text and image inputs
- To build an interactive and user-friendly web interface using **Streamlit**
- To provide structured vehicle details such as specifications, features, mileage, and maintenance advice
- To demonstrate the practical use of **Generative AI in real-world applications**
- To implement **secure API key handling** using environment variables
- To enable users to upload vehicle images and receive AI-based analysis
- To apply **prompt engineering techniques** to guide AI responses effectively
- To create a system that delivers **quick and meaningful AI-generated outputs**
- To gain hands-on experience in **AI model integration and deployment**
- To understand how **cloud-based AI services** can be used in application development
- To develop a solution that enhances **user experience through intelligent assistance**
- To explore the use of **multimodal AI (text + image processing)**
- To build a scalable foundation that can be enhanced with additional AI features in the future

3. TECHNOLOGIES USED

The AutoSage App is developed using a combination of modern technologies that enable the integration of Generative AI with a web-based interface. Each technology used in the project plays an important role in building and deploying the application.

1. Python

Python is the primary programming language used for developing the backend of the application. It is widely used in AI and machine learning projects due to its simplicity, readability, and strong library support.

Python controls the app flow and connects everything.

```
python

# Basic Python function to get response from Gemini

def get_gemini_response(prompt):
    response = model.generate_content(prompt)
    return response.text
```

Python was chosen for this project because it is the industry standard for Generative AI and Data Science.

- **Readability:** Python's "English-like" syntax makes it easy to write and maintain complex AI logic.
- **Rich Ecosystem:** Libraries like google-generativeai and Streamlit allow us to build a full-stack AI app in just a few dozen lines of code.
- **Rapid Prototyping:** Since Python is interpreted (executed line-by-line), we can test new prompts or UI changes instantly.

2. Streamlit

Streamlit is a Python framework used to build interactive web applications quickly and easily. It provides a user-friendly interface where users can enter text, upload images, and view AI-generated responses in real time.

Used to create the user interface.

```
python

import streamlit as st

st.title("🚗 AutoSage - Vehicle AI Assistant")

user_input = st.text_area("Ask a question about a vehicle")

uploaded_file = st.file_uploader("Upload vehicle image", type=["jpg", "png", "jpeg"])

if st.button("Get Response"):
    st.write("Processing your request...")
```

In this project, Streamlit handles the "Frontend" responsibilities, acting as the bridge between the user and the Gemini Flash model.

- **Input Collection:** It provides text areas for user queries and file upload widgets for vehicle images or PDFs.
- **State Management:** It handles the "Submit" button logic, triggering the backend functions only when the user is ready.
- **Real-time Feedback:** It displays "Processing..." spinners or progress bars while the Gemini model generates a response.
- **Output Formatting:** It renders the AI's response in clean, readable Markdown, including tables or bullet points for vehicle specs.

3. Google Generative AI (Gemini Flash)

Google’s **Gemini Flash** model is the core AI component of the project. It processes both text and image inputs and generates intelligent, structured responses. This model enables the application to act as a smart vehicle advisory assistant.

Used to load and interact with the AI model.

```
python

import google.generativeai as genai
import os
from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

model = genai.GenerativeModel("gemini-1.5-flash")
```

Feature	Application in Vehicle AI
Visual Reasoning	Identifies vehicle makes, models, or specific mechanical issues from a photo.
Data Extraction	Pulls structured data (like torque specs or oil types) from messy PDF manuals.
High Frequency	Handles rapid-fire questions from users without hitting "rate limits" quickly.
Agentic Vision	New in 2026, it can "investigate" an image by zooming in on specific details like serial numbers.

4. Python-Dotenv

The python-dotenv library is used to securely store and load the Google API key from a .env file. This ensures that sensitive information is not directly written into the source code.

Used to load the API key securely.

.env file

```
ini

GOOGLE_API_KEY=your_api_key_here
```

```
python

from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("GOOGLE_API_KEY")
```

In your app.py, you use python-dotenv to "inject" these secrets into your script at runtime.

```
import os
from dotenv import load_dotenv
import google.generativeai as genai

# Step 1: Search for the .env file and load variables into the environment
load_dotenv()

# Step 2: Retrieve the key using the os module
api_key = os.getenv("GOOGLE_API_KEY")

# Step 3: Configure the AI model
if api_key:
    genai.configure(api_key=api_key)
else:
    print("Error: GOOGLE_API_KEY not found in .env file!")
```

5. Pillow (PIL)

Pillow is a Python library used for basic image processing. It helps in handling and preparing uploaded vehicle images before sending them to the Gemini model for analysis.

Used to read and process uploaded images.

```
python

from PIL import Image

def input_image_setup(uploaded_file):
    if uploaded_file is not None:
        image = Image.open(uploaded_file)
        return image
```

Sending Image + Text to Gemini

```
def analyze_vehicle_image(prompt, uploaded_file):
    image = input_image_setup(uploaded_file)

    response = model.generate_content([prompt, image])
    return response.text
```

The Data Pipeline

1. **Streamlit:** Provides the file_uploader widget.
2. **Pillow:** Opens the file and converts it into an image object.
3. **Python Logic:** Bundles this image object with the user's text prompt.
4. **Gemini Flash:** Receives the data and performs the visual analysis.

This pipeline is designed for **multimodal** processing, meaning it handles both visual (image) and textual data simultaneously.

1. Streamlit: The Entry Point

2. Streamlit acts as the "receptionist" for your data. When a user interacts with the st.file_uploader widget, Streamlit captures the file as an **UploadedFile** object (which is a subclass of BytesIO).

- **Benefit:** It handles the complex file-streaming logic so you don't have to write custom HTML/JavaScript for the upload.

2. Pillow: The Visual Processor

Since the uploaded file is just a stream of bytes, we use **Pillow** to "open" it.

- **Action:** `Image.open(uploaded_file)` converts the raw bytes into a formatted image object.
- **Why?** This allows the script to verify the image is valid and ensures it is in a format (like RGB) that the Gemini model can interpret.

3. Python Logic: The Bundler

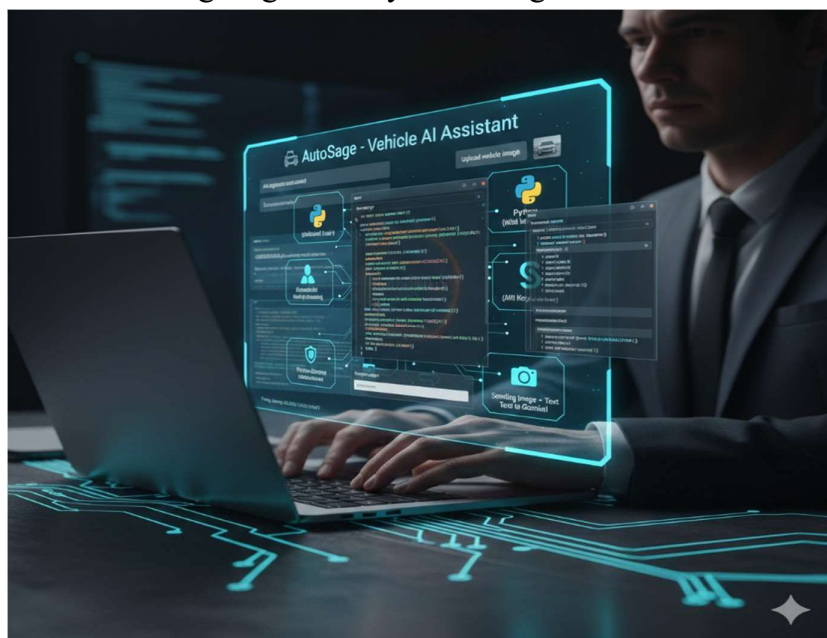
This is where the magic happens. Python takes two different pieces of data—the **text prompt** and the **image object**—and bundles them into a Python list.

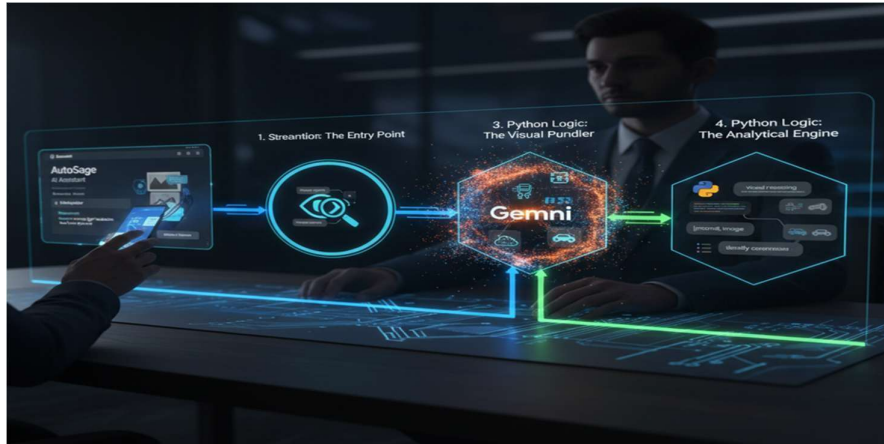
- **Flow:** It organizes the inputs so the Gemini API knows which text refers to which image.

4. Gemini Flash: The Analytical Engine

The bundled list is sent via an API call to **Gemini 1.5 Flash**.

- **Process:** The model analyzes the pixels of the image alongside the instructions in your text.
- **Result:** It generates a text-based response explaining what it saw (e.g., "The warning light in your image indicates low tire pressure").





4.SYSTEM ARCHITECTURE

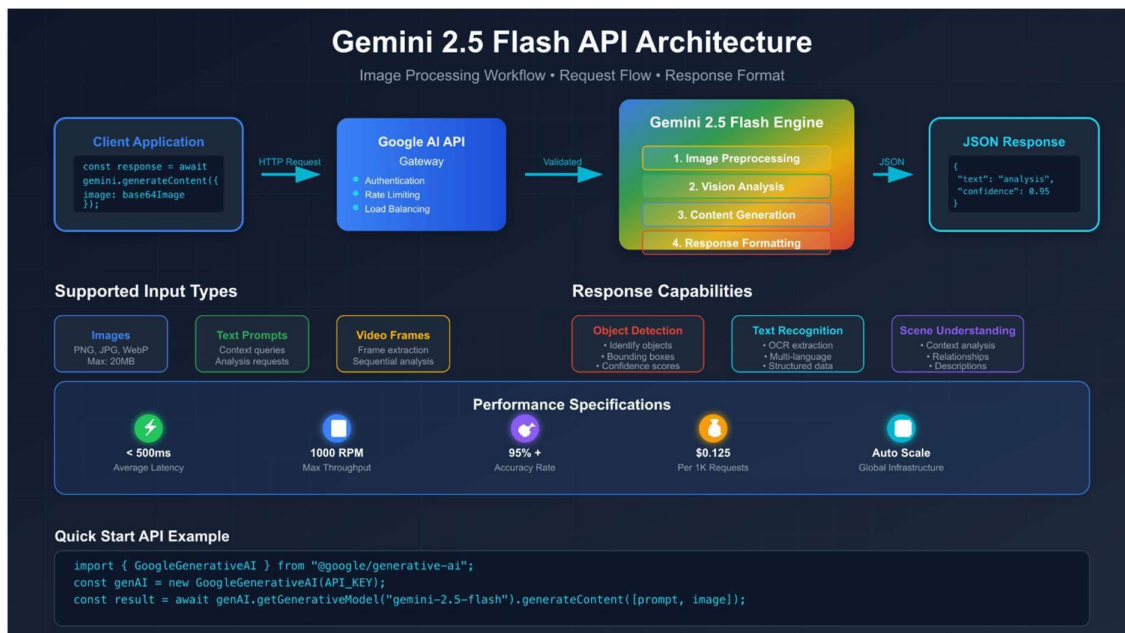
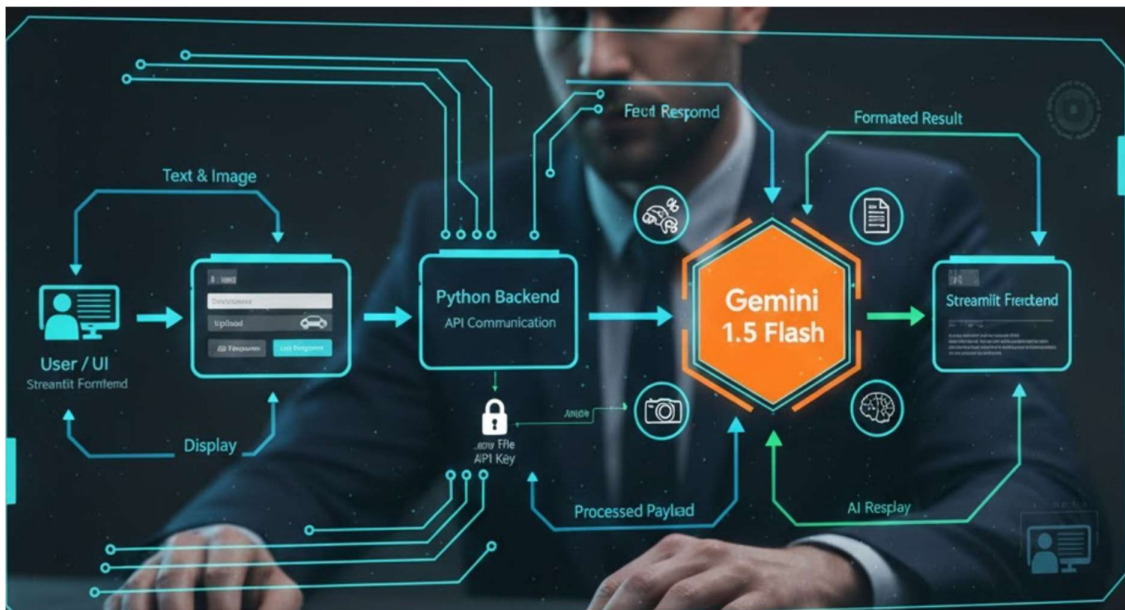
The AutoSage App follows a simple and efficient architecture that connects the user interface with the Gemini AI model to generate intelligent vehicle-related responses.

Working Flow of the System

The system operates in the following steps:

1. **User Interaction:**The user interacts with the application through the **Streamlit web interface**. The user can:
 - Enter a vehicle-related question in text form
 - Upload an image of a vehicle
2. **Input Processing:** The input provided by the user (text or image) is received by the **Python backend**.
 - Text input is prepared as a prompt
 - Image input is processed using the **Pillow library**
3. **API Communication** :The backend uses a securely stored **Google API key** to connect with the **Gemini Flash Generative AI model** via an API call.
4. **AI Model Processing:** The **Gemini Flash model** analyzes the input (text and/or image) and generates a structured and meaningful response related to vehicle details.
5. **Response Delivery** :The generated response is sent back to the **Streamlit application**, where it is displayed to the user in a readable format.

Architecture Diagram (Text Representation):



Key Components of the Architecture:

To ensure the AutoSage App operates reliably, each component has a specialized role. Below is a breakdown of the primary modules that make up our system architecture.

Component	Role
Streamlit UI	Acts as the frontend; collects user text/images and displays the final AI output.
Python Backend	The central orchestrator that processes data and manages the application logic.
Gemini Flash Model	The "Brain" of the app; generates intelligent vehicle insights and visual analysis.
API Key	The security token used to authenticate and authorize access to Google's AI services.
Pillow Library	Specifically handles image decoding and formatting for vehicle-related visuals.

Component Deep-Dive

- **Streamlit:** Chosen for its ability to create a "zero-code" frontend experience using only Python. It allows for rapid deployment of interactive widgets like file uploaders and text boxes.
- **Gemini 1.5 Flash:** Unlike larger models, the "Flash" variant is optimized for low latency. This is crucial for a vehicle assistant where users expect quick diagnostics or part identifications.
- **Security (python-dotenv):** By using an .env file, the **API Key** remains hidden from the main source code, preventing unauthorized access if the code is shared or hosted publicly.

5. Project Structure:

The project is organized in a simple and modular structure to ensure easy understanding, development, and maintenance. Each file and folder has a specific role in the application.

The **AutoSage App** is organized using a clean, modular directory structure that separates the core application logic from configuration and static resources. This ensures that the project is easy to maintain, scale, and deploy.

File Description

app.py

This is the core file of the project. It includes:

- Loading the Gemini Flash model
- Handling user input (text and image)
- Sending prompts to the AI model
- Displaying responses through Streamlit

requirements.txt

Contains all the necessary Python libraries required to run the project. This helps others easily set up the same environment.

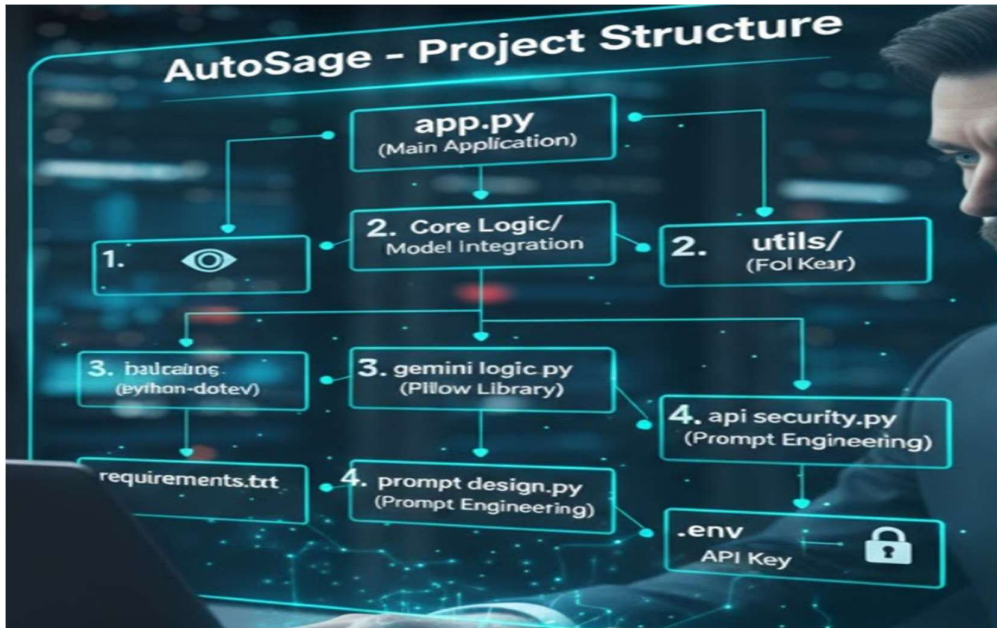
.env

Used to securely store the **Google API Key**. This ensures sensitive information is not hard-coded into the application.

README.md: Provides a high-level overview of the project, installation steps, and usage instructions for other developers.

assets/: A dedicated space for any local images, CSS files, or logos used to customize the Streamlit interface.

app.py: The heart of the project. This file contains the **Streamlit** code for the frontend, the **Pillow** logic for image handling, and the **Google Generative AI** integration.



6. REQUIREMENTS SPECIFICATION:

The successful development and execution of the AutoSage App requires a specific set of software libraries, hardware configurations, and API credentials. These requirements ensure that the application handles multimodal data (text and images) efficiently while maintaining a secure connection to the Google Cloud infrastructure.

Software Requirements

The software stack is designed to be lightweight yet powerful, focusing on the integration between local Python logic and cloud-based AI.

Software	Purpose
Python (3.8 or above)	The core engine used to orchestrate data flow and API calls.
Streamlit	The rapid-development framework used to build the interactive web UI.
Google Generative AI SDK	The official library used to communicate with the Gemini 1.5 Flash model.

Software	Purpose
Python-Dotenv	A security utility that injects secret keys from a .env file into the environment.
Pillow (PIL)	A robust imaging library used to decode and pre-process vehicle photos.

Hardware Requirements

Because the "heavy lifting" (AI processing) is handled by Google's servers, the local hardware requirements are quite modest, making AutoSage accessible on most modern laptops.

- **Processor:** Intel i3 or equivalent (Dual-core minimum).
- **RAM:** 4 GB (8 GB recommended for smoother multitasking during development).
- **Storage:** 500 MB free space (primarily for Python environments and temporary image caching).
- **Internet Connection:** Required. High-speed access is recommended for uploading high-resolution vehicle images to the Gemini API.

Python Libraries (requirements.txt):

Before launching the application, a virtual environment should be prepared with the following dependencies. These are the specific versions tested for compatibility with the Gemini 1.5/3.0 Flash series.

```
streamlit
google-generativeai
python-dotenv
Pillow
```

```
pip install -r requirements.txt
```

API Requirement & Security

The application cannot function without a handshake with Google's servers.

1. **Obtaining the Key:** A valid **Google API Key** must be generated via [Google AI Studio](#).
2. **Storage:** The key must never be placed directly in the app.py file. Instead, it must be stored in a .env file in the root directory.

```
GOOGLE_API_KEY=your_api_key_here
```



7. Google API Key Setup

To connect the AutoSage App with **Google's Gemini Flash model**, a valid **Google API Key** is required. This key authorizes the application to access Generative AI services.

Step 1: Generate the API Key

1. Go to **Google AI Studio**
 <https://aistudio.google.com/>
2. Sign in using your Google account.
3. Click "**Get API Key**"
4. Select "**Create API Key**"
5. Copy the generated key.

Step 2: Store API Key in .env File

Create a file named **.env** in your project folder and add:

```
GOOGLE_API_KEY=your_api_key_here
```

Step 3: Install dotenv Library

```
pip install python-dotenv
```

Step 4: Load API Key in Python

```
from dotenv import load_dotenv
import os

# Load environment variables from .env file
load_dotenv()

# Retrieve API key
api_key = os.getenv("GOOGLE_API_KEY")
```

Step 5: Configure Gemini API

```
import google.generativeai as genai

# Configure Gemini with API key
genai.configure(api_key=api_key)

# Load Gemini Flash model
model = genai.GenerativeModel("gemini-1.5-flash")
```

Final app.py Code – AutoSage App

```
import streamlit as st

import google.generativeai as genai

from PIL import Image

from dotenv import load_dotenv

import os

# -----

# Load API Key

# -----

load_dotenv()

genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

# Load Gemini Flash Model

model = genai.GenerativeModel("gemini-1.5-flash")

# -----

# Function to Process Uploaded Image

# -----
```

```

def input_image_setup(uploaded_file):
    if uploaded_file is not None:
        image = Image.open(uploaded_file)
        return image
    return None

# -----
# Prompt for AutoSage App
# -----
input_prompt = """
You are an automobile expert. Provide detailed and structured vehicle
information.

Include the following:
• Vehicle Type
• Specifications
• Key Features
• Mileage Information
• Maintenance Tips
• Approximate Price Range
"""

# -----
# Function to Get Gemini Response
# -----
def get_gemini_response(prompt, image=None):

```

```

    if image:
        response = model.generate_content([prompt, image])
    else:
        response = model.generate_content(prompt)
    return response.text

# -----
# Streamlit UI
# -----

st.set_page_config(page_title="AutoSage App", layout="centered")

st.title("🚗 AutoSage App using Gemini Flash")

st.write("AI-Powered Vehicle Information & Advisory System")

user_input = st.text_area("Ask a vehicle-related question:")

uploaded_file = st.file_uploader("Upload a vehicle image (optional)",
type=["jpg", "jpeg", "png"])

if st.button("Get AI Response"):
    if not user_input and not uploaded_file:
        st.warning("Please enter a question or upload an image.")
    else:
        image = input_image_setup(uploaded_file)
        with st.spinner("Generating response using Gemini Flash..."):
            response = get_gemini_response(input_prompt + user_input, image)
        st.subheader("🤖 AI Response")

```

```
st.write(response)
```

```
st.markdown("---")
```

```
st.caption("Developed as part of Google Cloud Generative AI Virtual Internship")
```

requirements.txt

```
streamlit  
google-generativeai  
python-dotenv  
Pillow
```

.env File

```
GOOGLE_API_KEY=your_api_key_here
```

Run the App

```
streamlit run app.py
```

8. Model Integration (Gemini Flash)

The core functionality of the AutoSage App depends on the integration of Google's Gemini Flash model, which is a powerful Generative AI model capable of understanding both text and images. This integration allows the system to generate intelligent, structured responses for vehicle-related queries.

Configuring the Gemini API:

Before using the Gemini model, the API must be configured using the securely stored Google API key.

```
import google.generativeai as genai
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Configure Gemini API
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
```

This step establishes a secure connection between the application and Google's Generative AI services.

Loading the Gemini Flash Model:

After configuring the API, the **Gemini Flash model** is initialized. This model is responsible for generating AI-based responses.

```
# Load Gemini Flash model
model = genai.GenerativeModel("gemini-1.5-flash")
```

The gemini-1.5-flash model is selected because it supports **fast response generation** and **multimodal input** (text + image).

Sending Input to the Model:

The model receives user input in the form of a text prompt and optionally an image. The Gemini model processes this input and returns a structured response.

```
def get_gemini_response(prompt, image=None):
    if image:
        response = model.generate_content([prompt, image])
    else:
        response = model.generate_content(prompt)
    return response.text
```

- If only text is provided, the model generates a response based on the prompt.
- If an image is also provided, the model uses both text and visual data for analysis.

Role of Gemini Flash in the Project

The Gemini Flash model enables the application to:

- Understand vehicle-related questions
- Analyze uploaded vehicle images
- Generate structured and informative responses

Act as an AI-powered vehicle advisory assistant

This integration forms the backbone of the AutoSage App, allowing it to function as an intelligent system rather than a static information provider.

Model Integration (Gemini Flash)

Integrating Google's Generative AI model for the AutoSage App

Step 1 Install Library

```
pip install google-generativeai
```

Step 2 Import Modules

```
import google.generativeai as genai
from dotenv import load_dotenv
import os
```

Step 3 Load API Key

```
.env load_dotenv()
api_key = os.getenv(
    "GOOGLE_API_KEY")
```

Step 4 Configure API

```
genai.configure_api
genai.configure(api_key=api_key)
```

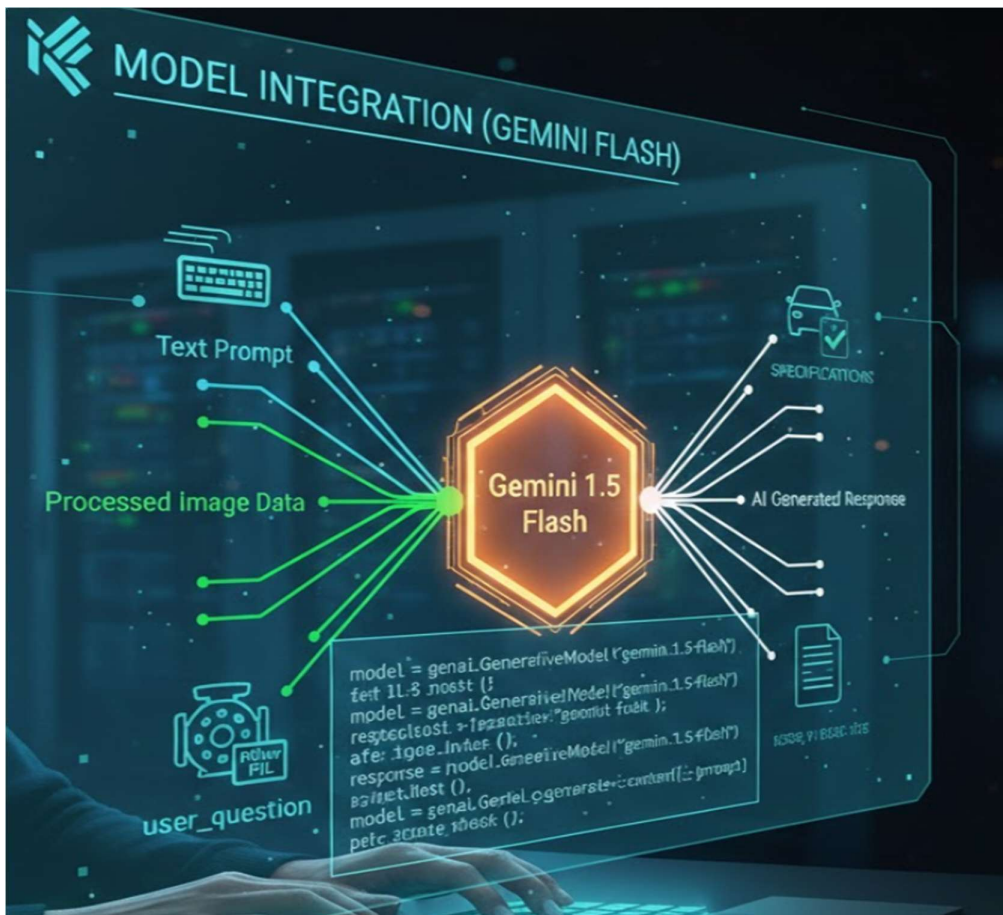
Step 5 Generate Response

```
response = model.generate_content(prompt, image)
```

Gemini Flash AI Model

- ✓ Generative AI
- ✓ Multimodal Input
- ✓ Gemini 1.5-flash
- ✓ Structured Responses

Google's Generative AI Model



9. Image Processing Function:

The AutoSage App allows users to upload vehicle images so that the Gemini Flash model can analyze them along with text input. To prepare images for AI processing, the Pillow (PIL) library is used.

Purpose of Image Processing

The image processing function ensures that the uploaded image is:

- Properly opened
- Converted into a format the Gemini model can read
- Ready to be sent along with the text prompt

Image Processing Code

```
from PIL import Image

def input_image_setup(uploaded_file):
    if uploaded_file is not None:
        # Open the uploaded image
        image = Image.open(uploaded_file)
        return image
    return None
```

How It Works

1. The user uploads an image through the Streamlit interface.
2. The file is passed to the `input_image_setup()` function.
3. The function uses Pillow to open and process the image.
4. The processed image is then sent to the Gemini model along with the prompt.

Using the Image with Gemini

```
image = input_image_setup(uploaded_file)
response = model.generate_content([prompt, image])
```

If no image is uploaded, the system will generate a response based only on the text input.

Importance in the Project

This function enables **multimodal AI capability**, allowing the application to:

1. Analyze vehicle images.
2. Provide visual-based insights.
3. Enhance the intelligence of responses.



10. Prompt Engineering:

Prompt engineering plays a crucial role in guiding the Gemini Flash model to generate accurate and structured responses. A well-designed prompt helps the AI understand the task and provide vehicle-related information in a clear format.

Purpose of Prompt Engineering

The goal of prompt engineering in the AutoSage App is to:

- Instruct the AI to behave like an automobile expert.
- Ensure responses are structured and informative.
- Focus on vehicle specifications, features, and advice.

Base Prompt Used in the Application

```
input_prompt = """
You are an automobile expert. Provide detailed and structured vehicle information.

Include the following:
• Vehicle Type
• Specifications
• Key Features
• Mileage Information
• Maintenance Tips
• Approximate Price Range
"""
```

This prompt ensures that every response from the Gemini model follows a consistent and useful format.

How the Prompt is Combined with User Input

The system combines the base prompt with the user's query before sending it to the model.

```
final_prompt = input_prompt + user_input
response = model.generate_content(final_prompt)
```

If an image is uploaded, the prompt is sent along with the image:

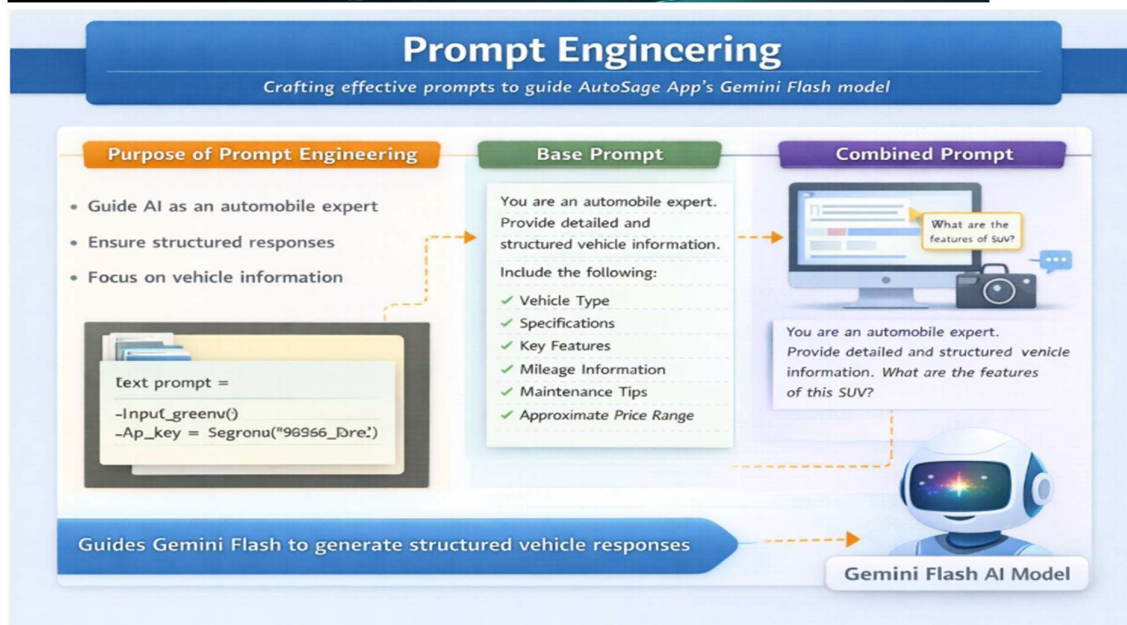
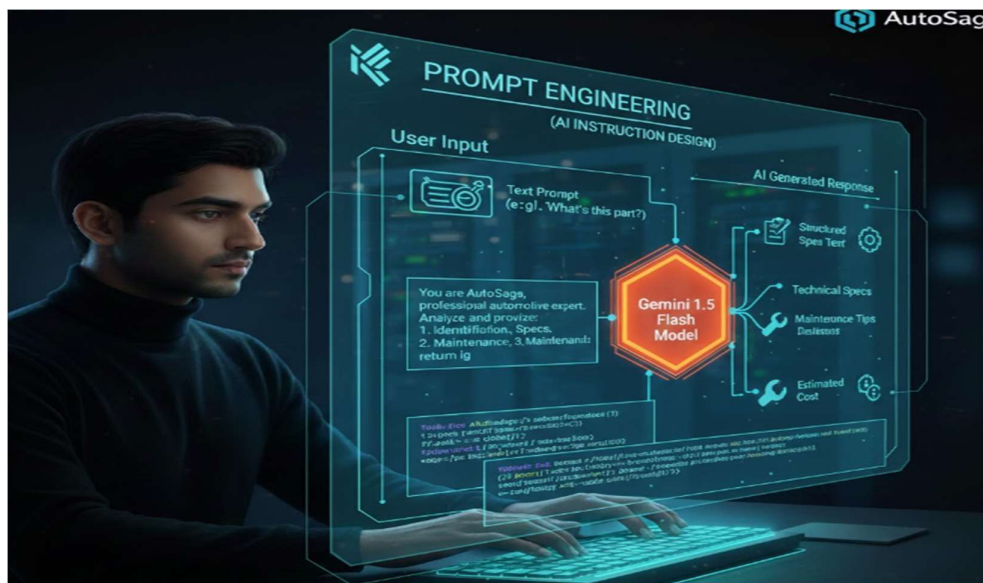
```
response = model.generate_content([final_prompt, image])
```

Importance of Prompt Design

Effective prompt design helps the system:

1. Generate relevant and structured responses.
2. Maintain consistency in AI output.
3. Improve clarity and usefulness of information.
4. Guide the AI toward vehicle-specific knowledge.

Prompt engineering transforms the Gemini model into a **specialized vehicle advisory assistant**.



11. Web Interface (Streamlit)

The AutoSage App uses Streamlit to create a simple, interactive, and user-friendly web interface. Streamlit allows the application to run as a web app without needing complex frontend development.

Purpose of Using Streamlit

Streamlit is used to:

- Create a web-based interface using Python
- Allow users to enter vehicle-related questions
- Enable users to upload vehicle images
- Display AI-generated responses in real time

Importing Streamlit

```
import streamlit as st
```

Setting Up the Page

```
st.set_page_config(page_title="AutoSage App", layout="centered")

st.title("🚗 AutoSage App Using Gemini Flash")
st.subheader("AI-Powered Vehicle Information & Advisory System")
```

User Input Field

This allows users to ask questions about vehicles.

```
user_input = st.text_input("Ask about any vehicle:")
```

Image Upload Option Users can upload vehicle images for AI analysis.

```
uploaded_file = st.file_uploader(
    "Upload a vehicle image (optional)",
    type=["jpg", "jpeg", "png"]
)
```

Submit Button

The button triggers the AI response generation.

```
submit = st.button("Get Vehicle Details")
```

Displaying the Response

```
if submit:
    if uploaded_file is not None:
        image_data = input_image_setup(uploaded_file)
        response = get_gemini_response(final_prompt, image_data)
    else:
        response = get_gemini_response(final_prompt)

    st.subheader("Vehicle Information:")
    st.write(response)
```

Role of Streamlit in the Project

Streamlit acts as the bridge between the user and the AI model by:

1. Collecting user input.
2. Sending data to the Gemini model.
3. Displaying AI-generated vehicle insights.

It enables the project to function as a **real-time AI-powered web application**.



12. Running the Application

After setting up the project files and installing the required libraries, the AutoSage App can be launched locally using Streamlit.

Install Required Libraries:

```
pip install -r requirements.txt
```

Verify API Key Setup

Make sure the .env file exists in your project folder:

```
GOOGLE_API_KEY=your_api_key_here
```

Run the Streamlit App

```
streamlit run app.py
```

Expected Terminal Output

```
You can now view your Streamlit app in your browser.
```

```
Local URL: http://localhost:8501
```

```
Network URL: http://192.168.x.x:8501
```

Open in Browser

Go to: <http://localhost:8501>

```
if st.button("Get AI Response"):
    if not user_input and not uploaded_file:
        st.warning("Please enter a question or upload an image.")
    else:
        image = input_image_setup(uploaded_file)
        final_prompt = input_prompt + user_input
        response = get_gemini_response(final_prompt, image)

        st.subheader("🖼️ AI Response")
        st.write(response)
```

13. Model Deployment

After developing and testing the AutoSage App locally, the next step is deployment. Deployment makes the application accessible to users through the internet instead of only running on a local machine.

Deployment Using Streamlit Cloud

Streamlit provides an easy way to deploy applications online.

Steps to Deploy:

1. Push your project to a GitHub repository
2. Go to <https://share.streamlit.io/>
3. Sign in using your GitHub account
4. Click “New App”
5. Select your repository and branch
6. Set the main file path as:

app.py

7. Click Deploy.

Adding API Key in Deployment

Since .env files are not uploaded to GitHub, you must add the API key in Streamlit Cloud settings.

1. Go to **App Settings** → **Secrets**
2. Add the key in this format:

```
GOOGLE_API_KEY = "your_api_key_here"
```

Deployment Strategies

Deploying your **AutoSage App** shifts it from a local script to a global tool. While Streamlit Community Cloud is the standard, several other platforms offer unique advantages depending on your scaling needs.

Platform	Best For	Key Advantage
Render	General web apps	Simple "set and forget" free tier for Python services.
Hugging Face Spaces	AI/ML Portfolio	Built-in community visibility; easy integration with other AI models.
Google Cloud (Cloud Run)	Scalable Enterprise	Best for high traffic; uses Docker for professional-grade stability.
AWS (App Runner)	Deep Ecosystem	Ideal if you plan to integrate other AWS services like RDS databases.

Deployment Workflow

Regardless of the platform, the path to "Production" follows a consistent logic:

1. **Local Development:** Code is written and tested locally using `streamlit run app.py`.
2. **Version Control:** Code is committed and pushed to a **GitHub** repository.
3. **Cloud Linkage:** The hosting platform (e.g., Render) is connected to your GitHub repo.
4. **Environment Configuration:** Your **Google API Key** is added to the platform's "Secrets" or "Environment Variables" section.
5. **Build & Launch:** The platform detects your `requirements.txt`, installs libraries, and launches the app live.

Benefits of Deployment

- **Universal Access:** Share a simple URL with anyone, anywhere, without them needing Python installed.
- **Mobile Readiness:** Users can take photos of their cars and get AI diagnostics directly from their smartphone browsers.

- **Portfolio Building:** A live link is significantly more impressive to recruiters than a folder of code.
- **Automated Updates:** Every time you push a fix to GitHub, your live website updates automatically.



14. Key Features

The AutoSage App provides several intelligent and user-friendly features by combining Generative AI with a web-based interface.

AI-Powered Vehicle Assistance

The application uses the **Gemini Flash model** to provide accurate and structured information about vehicles based on user queries.

Multimodal Input Support

Users can:

- Ask questions using text
- Upload vehicle images

The system processes both inputs to generate more meaningful responses.

Structured Vehicle Information

The AI provides details such as:

- Vehicle specifications
- Key features
- Mileage information
- Maintenance advice
- Approximate price range

Interactive Web Interface

Built with **Streamlit**, the app offers:

- Simple input fields
- Image upload option
- Instant AI-generated responses

Secure API Integration

The application uses environment variables to securely store the **Google API Key**, ensuring safe communication with Gemini services.

Real-Time Response Generation

The Gemini Flash model generates responses quickly, making the application feel interactive and responsive.

Easy Deployment

The app can be easily deployed using platforms like **Streamlit Cloud**, making it accessible from anywhere.

These features make the AutoSage App an effective **AI-powered vehicle advisory assistant**.

Feature	Input Type	Primary User Benefit
Visual Diagnosis	Image	Immediate clarity on mechanical issues.
Expert Q&A	Text	Access to professional-grade advice 24/7.
Tech Specs	Text/Image	Fast data retrieval for buying or repairs.
Maintenance Tips	Text	Extends vehicle lifespan and saves money.
Market Insights	Text	Budgeting confidence for sales and service.

15. Future Enhancements

While the AutoSage App successfully demonstrates the integration of Generative AI with a web application, there are several improvements that can be added in the future to enhance its functionality and user experience.

Chat History Feature

The application can be upgraded to store previous conversations so users can view their earlier queries and responses.

Voice Input Support

Voice-based interaction can be added to allow users to ask vehicle-related questions using speech instead of typing.

Vehicle Comparison Tool

A comparison feature can be implemented where users can compare two or more vehicles based on specifications, mileage, and features.

Downloadable Vehicle Reports

The system can generate a downloadable PDF report containing vehicle details and recommendations.

Integration with Live Databases

Connecting the app to real-time automobile databases can provide more accurate and up-to-date vehicle information.

User Login System

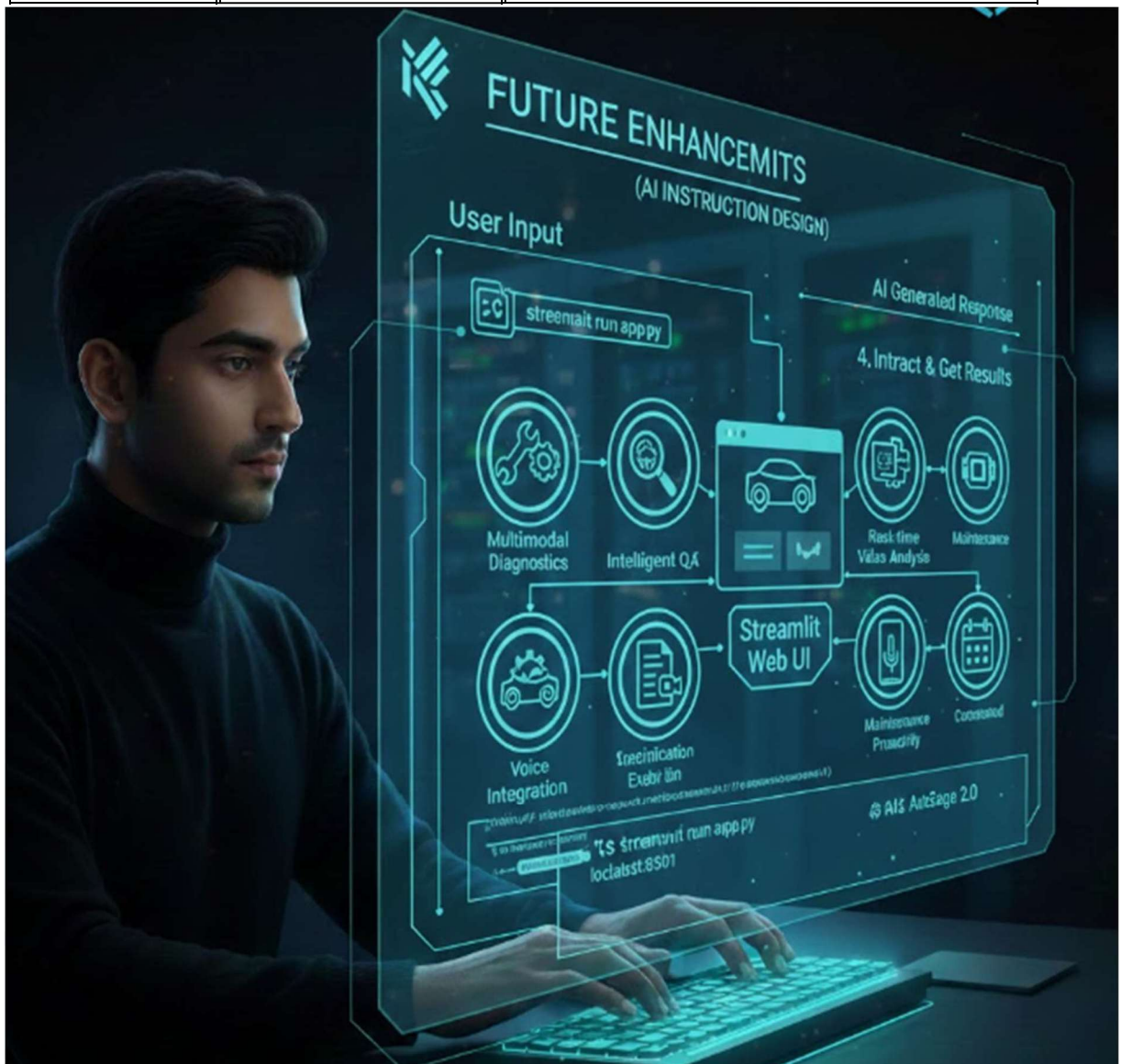
A login system can be introduced to personalize the experience and save user preferences or history.

Multilingual Support

The app can be enhanced to support multiple languages, making it accessible to a wider audience.

These enhancements can make the AutoSage App more powerful, user-friendly, and suitable for real-world deployment.

Phase	Enhancement	Expected Impact
Short Term	Voice Commands	Safer in-car utility and better accessibility.
Medium Term	OBD-II Integration	100% accurate, real-time diagnostic data.
Long Term	Predictive AI	Prevention of breakdowns and 30% lower repair costs.



16. Conclusion

The **AutoSage App using Gemini Flash** successfully demonstrates how **Generative AI** can be integrated into a web application to create an intelligent and interactive system. By combining **Google's Gemini Flash model** with a **Streamlit-based interface**, the project provides users with structured and meaningful vehicle-related information based on both text queries and uploaded images.

This project highlights the practical application of **Google Cloud Generative AI** in solving real-world problems. It showcases how AI models can be guided using prompt engineering to deliver domain-specific assistance—in this case, acting as an automobile advisory assistant.

Through the development of this project, valuable hands-on experience was gained in:

- Integrating AI APIs into applications
- Handling multimodal inputs (text and image)
- Designing user-friendly interfaces
- Deploying AI-powered web applications

The AutoSage App reflects the growing potential of AI-driven solutions and demonstrates how modern AI technologies can improve user experience by providing quick, structured, and intelligent responses.



